

Multiscale Estimation of Intrinsic Dimensionality of Point Cloud Data and Multiscale Analysis of Dynamic Graphs

Senior Thesis

Jason Lee

Advisor: Mauro Maggioni

April 30, 2010

Contents

1	Multiscale Estimation of Intrinsic Dimensionality of Point Cloud Data	5
1.1	Introduction	5
1.2	Multiscale SVD for Dimension Estimation	6
1.2.1	Model	6
1.2.2	Linear Manifolds	6
1.2.3	Nonlinear Case and Curvature	6
1.3	Multiscale SVD Algorithm	7
1.4	Experimental Results	7
1.4.1	k-dimensional sphere	7
1.4.2	Manifold Clustering	8
1.4.3	Proposed Algorithm for Manifold Clustering	10
1.4.4	Comparison with other Dimension Estimation Algorithms	10
1.5	Future Work	12
2	Multiscale Analysis of Dynamic Networks	13
2.1	Introduction	13
2.1.1	Pervasiveness of Dynamic Networks	13
2.1.2	Previous Work	13
2.1.3	Difficulties of Comparing Graphs	13
2.1.4	Motivation for Multiscale Graph Analysis	14
2.2	Multiscale Analysis Methodology	15
2.2.1	Notation	15
2.2.2	Random Walk on Graphs	16
2.2.3	Downsampling the Graph	16
2.2.4	Diffusion Time	16
2.2.5	Multiscale Measurements of the Dynamics	17
2.2.6	Description of the Algorithm	18
2.3	Simulation Results	18
2.3.1	Construction of Graphs	18
2.3.2	Adding Vertices and Edges	19
2.3.3	Updating of the Clusters	19
2.3.4	Description of the Experiment	19
2.3.5	Analysis of the Dynamics	20
2.3.6	Interplay between Diffusion Time and Dynamics	22
2.4	Experimental Results	23
2.4.1	Building the Graph	23
2.4.2	Multiscale Analysis Algorithm	23
2.4.3	Analysis	24
2.5	Future Work	24

Chapter 1

Multiscale Estimation of Intrinsic Dimensionality of Point Cloud Data

1.1 Introduction

The problem of estimating the intrinsic dimensionality of a point cloud is of interest in a wide variety of problems. To cite some important instances, it is equivalent to estimating the number of variables in a statistical linear model in statistics, the number of degrees of freedom in a dynamical system, the intrinsic dimensionality of a data set modeled by a probability distribution highly concentrated around a low-dimensional manifold. Many applications and algorithms crucially rely on the estimation of the number of components in the data, for example spectrometry, signal processing, genomics, economics, to name only a few. Finally, many manifold learning algorithms assume that the intrinsic dimensionality is given.

When the data is generated by a multivariate linear model, $x_i = \sum_{j=1}^k \alpha_j v_j$ for some random variables α_j , and the v_j 's are fixed vectors in $\mathbb{R}^D$, the number of components is simply the rank of the data matrix X , where X_{ij} is the j -th coordinate of the i -th sample. In this case principal component analysis (PCA) or singular value decomposition (SVD) may be used to recover the rank and the subspace spanned by the v_j 's, and this situation is well understood, at least when the number of samples n goes to infinity. The more interesting case in most applications assumes that we observe noise measurements $\tilde{x}_i = x_i + \eta_i$, where η represents noise. This case is also quite well understood, at least when n tends to infinity (see e.g., out of many works, [10], [20], [23] and references therein).

The finite sample situation is less well understood. While we derive new results in this direction with a new approach, the situation in which we are really interested is that of data having a geometric structure more complicated than linear. In particular, an important trend in machine learning and analysis of high-dimensional data sets assumes that data lies on a nonlinear low-dimensional manifold. Several algorithms have been proposed to estimate intrinsic dimensionality in this setting; for lack of space we cite only [14] [9], [5], [4], [7], [22], [8] and we refer the reader to the many references therein. One the most important take away messages from the papers above is that most methods are based on estimating volume growth rate of an approximate ball of radius r on the manifold, and that such techniques necessarily require a number of sample points exponential in the intrinsic dimension. Moreover, such methods do not consider the case when high-dimensional noise is added to the manifold. The methods of this paper, on a restricted model of “manifold+noise”, only requires a number of points essentially proportional to the intrinsic dimensionality, as detailed in [15].

1.2 Multiscale SVD for Dimension Estimation

1.2.1 Model

We start by describing a stochastic geometric model generating the point clouds we will study. Let $(\mathcal{M}, g)$ be a smooth k -dimensional Riemannian manifold with bounded curvature, isometrically embedded in $\mathbb{R}^D$. Let η be $\mathbb{R}^D$ -valued with $\mathbb{E}[\eta] = 0$, $\text{Var}[\eta] = \sigma^2$ (the “noise”), for example $\eta \sim \sigma\mathcal{N}(0, I_D)$. Let $X = \{x_i\}_{i=1}^n$ be a set of uniform (with respect to the natural volume measure on $\mathcal{M}$) Our observations $\tilde{X}$ are noisy samples: $\tilde{X} = \{x_i + \sigma\eta_i\}_{i=1}^n$, where η_i are i.i.d. samples from η and $\sigma > 0$. These points may also be thought of as sampled from a probability distribution $\tilde{\mathcal{M}}$ supported in $\mathbb{R}^D$, concentrated around $\mathcal{M}$. Here and in what follows we represent a set of n points in $\mathbb{R}^D$ by an $n \times D$ matrix, whose (i, j) entry is the j -th coordinate of the i -th point. In particular, X and $\tilde{X}$ will be used to denote both the point cloud and the associated $n \times D$ matrices, and N is the noise matrix of the η_i ’s.

The problem we consider is to **estimate** $k = \dim \mathcal{M}$, **given** $\tilde{X}$.

1.2.2 Linear Manifolds

Let us first consider the case when $\mathcal{M}$ is a *linear manifold* (e.g. the image of a cube under a linear map), and η is Gaussian noise; the standard approach is to compute the principal components of $\tilde{X}$, and use the singular values to estimate k . Let $\text{cov}(X) = \frac{1}{n}X^TX$ be the $D \times D$ covariance matrix of X , and $\Sigma(X) = (\sigma_i^2)_{i=1}^D$ its eigenvalues. These are the singular values (S.V.) squared of $n^{-1/2}X$, i.e. the first D diagonal entries of Σ in the singular value decomposition (SVD) $X = U\Sigma V^T$. At least for $n \sim k \log k$, it is easy to show that with high probability (w.h.p.) exactly k S.V.’s are nonzero, and the remaining $D - k$ are equal to 0.

Since we are given $\tilde{X}$ and not X , we think of $\tilde{X}$ as a random perturbation of X and expect that $\Sigma(\tilde{X})$ is close to $\Sigma(X)$, so that $\sigma_1, \dots, \sigma_k \gg \sigma_{k+1}, \dots, \sigma_D$, allowing us to estimate k correctly with high probability (w.h.p.). This is a very common procedure, applied in a wide variety of situations, and often generalized to kernelized versions of principal component analysis, such as widely used dimensionality reduction methods.

1.2.3 Nonlinear Case and Curvature

In the general case when $\mathcal{M}$ is a *nonlinear manifold*, there are several problems with the above line of reasoning. Curvature in general forces the dimensionality of a best-approximating hyperplane, such as that provided by a truncated SVD, to be much higher than necessary. As a first trivial example, consider a planar circle ($k = 1$) embedded in $\mathbb{R}^D$: $\text{cov}(X)$ has exactly 2 nonzero eigenvalues equal to the radius squared. More generally, it is easy to construct a one-dimensional manifold ($k = 1$) in $\mathbb{R}^D$ such that $\text{cov}(X)$ has full rank D : it is enough to pick a curve that spirals out in more and more dimensions. The above phenomena are a consequence of performing SVD globally: if one thinks locally, the matter seems once again easily resolved. Let $z \in \mathcal{M}$, r a small enough radius and consider only the points $X^{(z,r)}$ in $B_z(r) \cap \mathcal{M}$ (the ball is in $\mathbb{R}^D$). For r small enough, $X^{(z,r)}$ is well-approximated by a portion of k -dimensional tangent plane $T_z(\mathcal{M})$, and therefore we expect $\text{cov}(X^{(z,r)})$ to have k large eigenvalues, growing like $O(r^2)$, and smaller eigenvalues caused by curvature, growing like $O(r^4)$. By letting $r_z \rightarrow 0$, i.e. *choosing r_z small enough dependent on curvature*, the curvature eigenvalues tend to 0 faster than the top k eigenvalues. Therefore, if we were given X , in the limit as $n \rightarrow \infty$

and $r_z \rightarrow 0$, this would give a consistent estimator of k . It is important to remark that these two limits are not unconditional: we need r_z to approach 0 slow enough and n to grow fast enough so that $B_z(r)$ contains enough points to accurately estimate the SVD in $B_z(r)$.

However, more interestingly, if we are given $\tilde{X}$, i.e. noise is added to the samples, we meet another constraint, that forces us to not take r_z too small. Let $\tilde{X}^{(z,r)}$ be $\tilde{\mathcal{M}} \cap B_z(r)$. If r_z is comparable to a quantity dependent on σ and D (for example, $r_z \sim \sigma\sqrt{D}$ when $\eta \sim \sigma\mathcal{N}(0, I_D)$), and $B_z(r)$ contains enough points, $\text{cov}(\tilde{X}^{(z,r)})$ may approximate the covariance of η rather than that of points on $T_z(\mathcal{M})$. Only for r_z larger than a quantity dependent on σ , yet smaller than a quantity depending on curvature, conditioned on $B_z(r)$ containing enough points, will we expect $\text{cov}(\tilde{X}^{(z,r)})$ to approximate a “noisy” version of $T_z(\mathcal{M})$.

Since we are not given σ , nor the curvature of $\mathcal{M}$, we cannot determine quantitatively the correct scale $r = r(z)$ qualitatively described above. This suggests that we take a *multiscale approach*. For every point $z \in \mathcal{M}$ and scale parameter $r > 0$, let $\text{cov}(z, r) = \text{cov}(\tilde{X}^{(z,r)})$ and let $\{\sigma_i^{(z,r)}\}_{i=1,\dots,D}$ be the corresponding eigenvalues, as usual sorted in nonincreasing order. We will call them multiscale singular values (S.V.’s) [21]. What is the behavior of $\sigma_i^{(z,r)}$ as a function of i, z, r ?

1.3 Multiscale SVD Algorithm

The reasoning for the linear case combined with studying the affect of curvature on the $\sigma_i^{(z,r)}$ suggests that the $\sigma_i^{(z,r)}$ corresponding to tangential singular vectors grow linearly in r and curvature singular values grow quadratically in r . These observations are formalized in [15] and [16].

- (1) **compute** $\sigma_i^{(z,r)}$, for each $z \in \mathcal{M}$, $r > 0$, $i = 1, \dots, D$.
- (2) **estimate the noise size** σ , obtained from the bottom S.V.’s which do not grow with r . Split the S.V.’s into noise S.V.’s and non-noise S.V.’s.
- (3) **identify a range of scales** where the noise S.V.’s are small compared to the other S.V.’s.
- (4) **estimate**, in the range of scales identified, which S.V.’s, among the non-noise S.V.’s, correspond to tangent directions and which ones correspond to curvatures, by comparing the growth rate as being linear or quadratic in r^2 .

1.4 Experimental Results

1.4.1 k-dimensional sphere

The first example is a 10-dimensional sphere embedded in 15 dimensions corrupted with noise $\sigma = .1$ in every dimension. We sample 1000 points from this distribution uniformly at random and perform the Multiscale SVD algorithm. The results are in Figure 1.1.

A 10-dimensional sphere lies is naturally embedded in $\mathbb{R}^{11}$ which spans 2^{11} orthants. Thus we are sampling, on average, $\frac{1}{2}$ a point every orthant. Furthermore, the points are being corrupted by gaussian noise. In this case, we correctly estimate the sphere as 10-dimensional on 999 out of 1000 samples. From the multiscale singular value plot 1.1, it

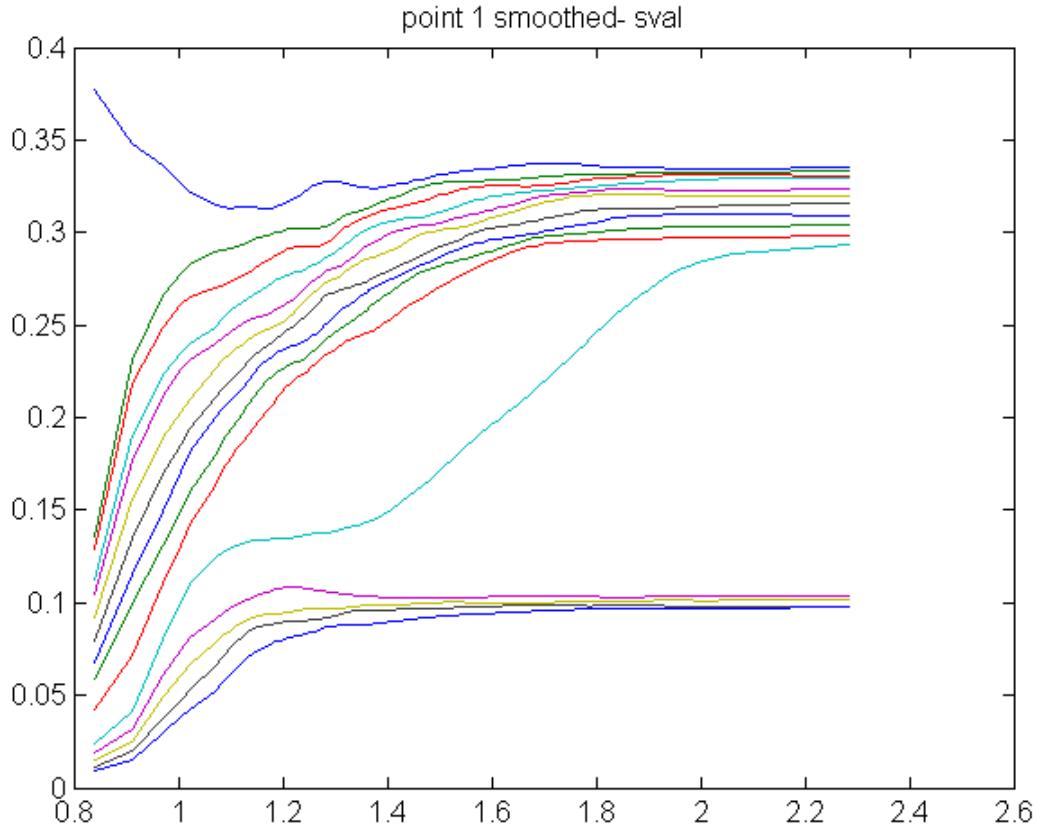


Figure 1.1: $\mathbb{S}^{10}$ embedded in $\mathbb{R}^{15}$ corrupted with noise. Plot of multiscale singular values.

is easy to identify 10 linearly growing singular values and the 11th quadratically growing singular value due to curvature. At the very small scales, the singular values cannot be separated, but at middle scales there is a clear gap between the 10th and 11th singular values. At large scales, there are 11 large singular values and the bottom 4 converge at the noise level.

1.4.2 Manifold Clustering

Since the Multiscale SVD algorithm provides pointwise dimension estimates combined with a range of scales where the manifold is well-approximated by linear manifolds, we can use it for manifold clustering. So far, we have only explored it for clustering manifolds of different dimension. We later propose an algorithm for clustering manifolds in the general case. Figure 1.2 shows an example of clustering by dimension. The dataset consists of 500 points sampled from a 5-dimensional sphere and 500 points sampled from a 4-dimensional plane corrupted with noise $\sigma = .1$ embedded in $\mathbb{R}^{15}$. The red points are estimated to be dimension 4, blue points are estimated to be dimension 5, and cyan points are estimated to be dimension 6. In this example, 95% of the points are correctly classified, demonstrating the robustness of our method. This can be easily improved if our algorithm was tailored for manifold clustering.

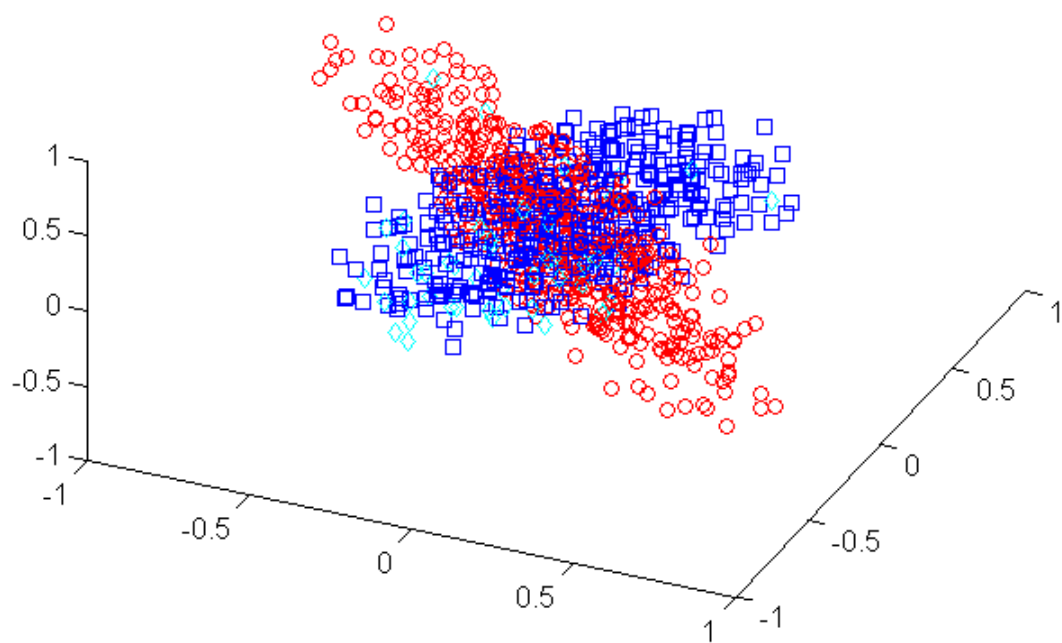


Figure 1.2: 5-dimensional sphere and 4-dimensional plane corrupted with $\sigma = .1$

1.4.3 Proposed Algorithm for Manifold Clustering

The pointwise dimension estimation algorithm can be used without any modification for separating manifolds into different dimensions. Here we discuss an algorithm that clusters manifolds of the same dimension. The primary output of the Multiscale SVD algorithm is a dimensions estimation and a scale r where the tangent plane faithfully approximates the manifold. The algorithm selects such a maximal scale for each point and returns this. For nearby points on the same manifold, these tangent planes will be similar. For nearby points on different manifolds, there tangent planes will be very different. For points on the same manifold but not close to each other, the tangent planes will also be very different. This line of reasoning motivates the following algorithm.

Algorithm 1 (Manifold Clustering). *1. Build affinity matrix $W_{ij} = \exp(-\frac{\|x_i - x_j\|^2}{\sigma_1^2} - \frac{a(x_i, y_i)^2}{\sigma_2^2})$. $a(x_i, y_i) = \sin \Theta$ where Θ is the principal angle between the maximal tangent planes at x_i and x_j .*

2. Perform spectral clustering on W . Estimate the number of clusters from the spectrum.

There have been some preliminary tests using this algorithm and appears successful. An open question is how to choose the parameters σ_1 and σ_2 and under what conditions are we guaranteed recovery. Recent work along the same lines includes [1] and references therein.

1.4.4 Comparison with other Dimension Estimation Algorithms

Let the set $\mathbb{S}^k(D, n, \sigma)$ be n i.i.d. points uniformly sampled from the unit k -sphere embedded in $\mathbb{R}^D$ ($D \geq k + 1$), with added noise $\sigma\mathcal{N}(0, I_D)$. We let $\mathbb{Q}^k(D, n, \sigma)$ be n i.i.d. points uniformly sampled from the unit k -dimensional cube embedded in $\mathbb{R}^D$ ($D \geq k$), with added noise $\sigma\mathcal{N}(0, I_D)$.

Our test sets consists of: $\mathbb{Q}^k(d, n, \sigma)$ with $(k, d) = \{(5, 10), (5, 100)\}$ and for each such choice, any combination of $n = 500, 100$, $\sigma = 0, 0.01, 0.05, 0.1, 0.2$; $\mathbb{S}^k(d, n, \sigma)$ with $k = \{5, 9\}$ and the other parameters as for the cube. We therefore have a portion of a linear manifold (the cube) and a manifold with simple curvature (the sphere). We considered several other cases, with more complicated curvatures, with similar results. We compare our algorithm with the algorithms of [9], [5], [4], see Figure 1.3. It is apparent that all the algorithms that we tested against are at the very least extremely sensitive to noise, and in fact they do not seem reliable even in the absence of noise. We are not surprised by such sensitivity, however we did try hard to optimize the parameters involved. First of all we note that our algorithm has no parameters besides $\tilde{X}$. The algorithms we are comparing it to have several parameters (from 4 to 7), which we tuned after rather extensive experimentation, relying on the corresponding papers and commentary in the code. In Figure 1.3 we report the mean, minimum, and maximum estimated dimensionality by each algorithm, upon varying the parameters of the algorithm. In most cases, this did not improve their performance significantly. We report mean, minimum and maximum dimensionality estimates over 5 runs.

We interpret these experiments, also in light of the theoretical results of [16], as a consequence that our algorithm, unlike the others considered, is not volume-based. The estimation of volumes is expected to require a number of samples exponential in the dimensionality of $\mathcal{M}$ []. The results in this paper, and their considerable refinements in

[16] for the finite sample case, support the intuition that the presented technique only requires a number of points linear in the dimensionality of $\mathcal{M}$.

We also ran our algorithm on the MNIST data set: for each digit from 0 to 9 we extracted 2000 random points, applied the algorithm, and projected them onto their top 80 principal components to regularize the search for nearest neighbors. The estimated intrinsic dimensionalities were: 2, 3, 5, 3, 5, 3, 2, 4, 3. However, the plots of multiscale singular values is not consistent with the hypothesis that the data is generated as a low-dimensional manifold plus noise. In particular, for several digits the intrinsic dimensionality depends on scale.

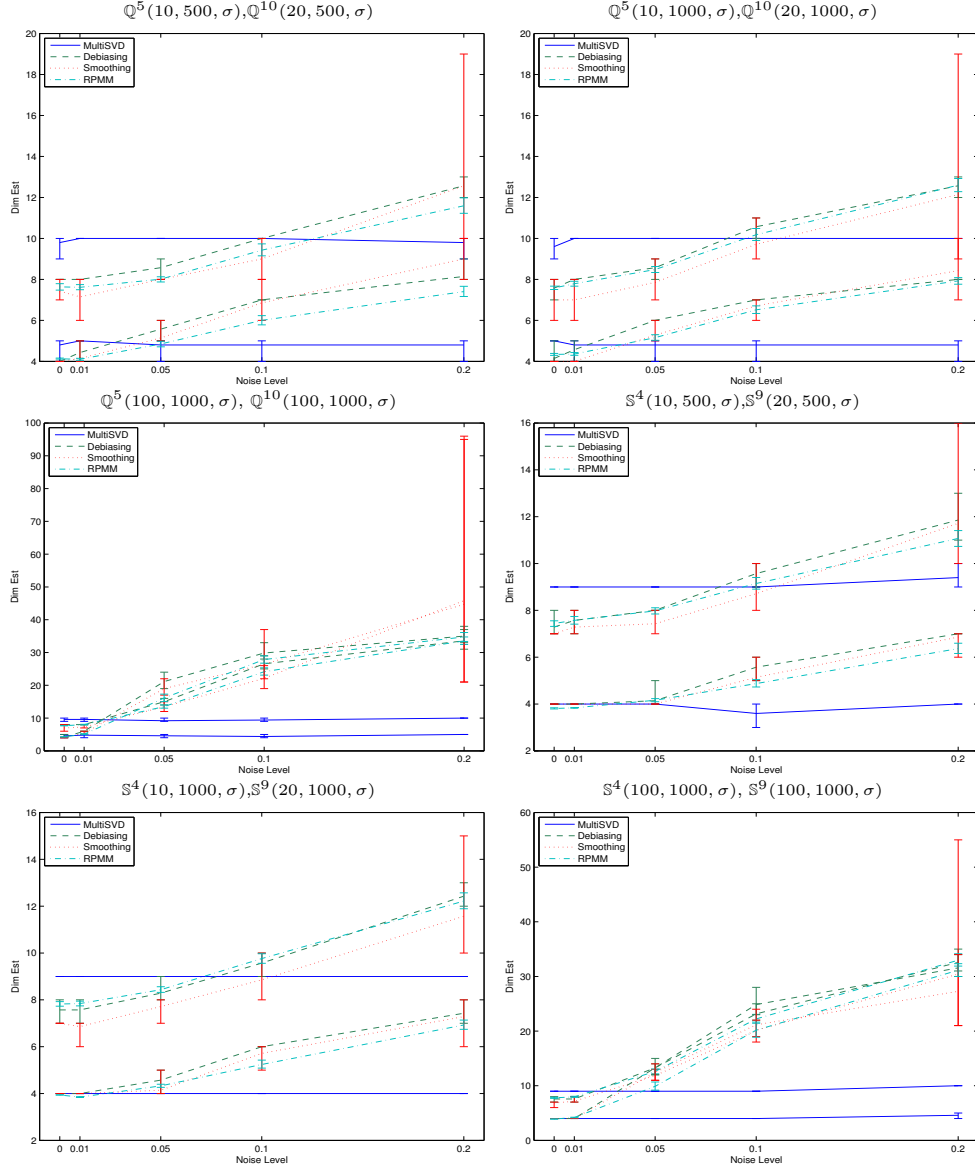


Figure 1.3: Benchmark data sets, comparison between our algorithm and “Debiasing” [4], “Smoothing” [5] and RPMM in [9] (with choice of the several parameters in those algorithm that seem optimal). $\mathbb{Q}^k(D, n, \sigma)$ consists of n points uniformly sampled from the k -dimensional cube embedded in D dimensions, and corrupted by $\eta \sim \sigma\mathcal{N}(0, I_D)$. $\mathbb{S}^k$ is a k -dimensional sphere. We report mean, minimum and maximum values of the output of the algorithms over several realizations of the data and noise. The horizontal axis is the size of the noise, the vertical is the estimated dimension. Even without noise current state-of-art algorithm do not work very well, and when noise is present they are unable to tell the noise from the intrinsic manifold structure. Our algorithm shows great robustness to noise.

1.5 Future Work

We presented an algorithm based on multiscale geometric analysis via principal components, that can very effectively and in a stable way estimate the intrinsic dimensionality of point clouds generated by perturbing low-dimensional manifolds by high-dimensional noise. In [16] it is shown that the algorithm works with high probability. Moreover, under reasonable hypotheses on the manifold $\mathcal{M}$ and the size of the noise, the probability of success is high as soon as the number of samples n is proportional to the intrinsic dimension k of $\mathcal{M}$. It is therefore a manifold-adaptive technique.

Future research directions include adapting the algorithm for hybrid linear modeling, manifold clustering, motion segmentation, dictionary learning, and semi-supervised learning. These problems benefit from the multiscale geometric analysis introduced in this paper.

Chapter 2

Multiscale Analysis of Dynamic Networks

2.1 Introduction

2.1.1 Pervasiveness of Dynamic Networks

This work focuses on studying properties of dynamic networks including online social networks (Facebook, Twitter, LiveJournal etc.), instant messaging networks (MSN, AIM, Skype etc.), game networks (various MMORPGs), p2p networks (bittorrent, Gnutella) and citation networks. These networks are constantly evolving in time with nodes entering and leaving and edges appearing and disappearing.

Our goal is to analyze network dynamics with applications to social network analysis, anomaly detection, and prediction/inference on networks; we develop a multiscale analytical approach to analyzing dynamics of graphs.

2.1.2 Previous Work

Over the past decade, there has been considerable interest in structural properties of networks. Starting with the power-law model [3], researchers have discovered small diameter laws and power law degree distributions that exist in many different networks. Numerous explanations have been proposed to explain this phenomenon and numerous generative models generate static graphs with these properties.

Even more recently, network analysis researchers have proposed generative models that obey static patterns and mimic temporal evolution patterns. A recent model based on the Kronecker product [13] produces graphs with desirable structural properties: heavy-tailed distribution and diameter, but also match the temporal behavior of real networks.

Instead of modeling dynamic networks, our approach is to analyze the dynamics of networks. Our approach is multiscale which is justified by significant research that shows social networks exhibit clustering or community properties. Furthermore, the success of Kronecker models that evolve similarly to real models suggests that dynamics occur in a multiscale fashion.

2.1.3 Difficulties of Comparing Graphs

It is very difficult to perform time series analysis on graphs because there is no agreed upon method to compare two different graphs. In the Computer Science Theory literature,

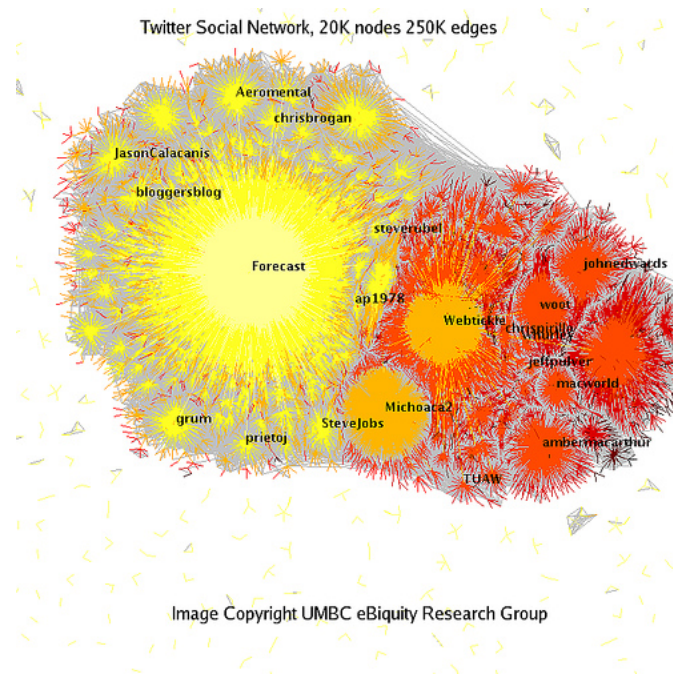


Figure 2.1: Snapshot of Twitter. Notice the clustering.

the notion of graph isomorphism is used to define graph equivalence. Graph isomorphism is a combinatorial notion and not flexible enough for our purposes. For example, if two graphs differ by ϵ on any edge weight they are not isomorphic. There is no notion of closeness or a metric given by Graph Isomorphism. Graph Isomorphism also has no known polynomial time algorithm and is not practical to compute for even moderately sized graphs (1000+ nodes).

The Social Network analysis community has developed tools such as degree-distribution, clustering coefficient, radius and diameter. These methods are fast to compute and do allow comparison between two graphs of different sizes; however, they are global in nature and not discriminative. Many different class of graphs exhibit the same diameter and degree distribution, but are very different. The networking community has developed several models that generate graphs with similar degree distributions, but are very different. This indicates that such global measures cannot discriminate between very different network models. See [3, 19].

2.1.4 Motivation for Multiscale Graph Analysis

Our goal is to perform a time series analysis on graphs; we develop a method that can distinguish between graphs with different number of nodes, different number of edges and different edge weights. For example, the Facebook graph before person 1,000,000 joins and after 1,000,000 joins should not have changed much, unless this person forms many friendships with two very different groups of people.

In addition to measuring changes between two graphs, we are able to reveal the portions of the graph undergoing the changes through a multiscale decomposition of the dynamics. A large perturbation to the graph will exhibit a difference on the coarse scale and a small perturbation will exhibit a fine scale change. This motivates the need for a multiscale procedure to measure differences between graphs. Furthermore, many social networks exhibit a natural multiscale structure. See Figure 2.2 for a multiscale decomposition.

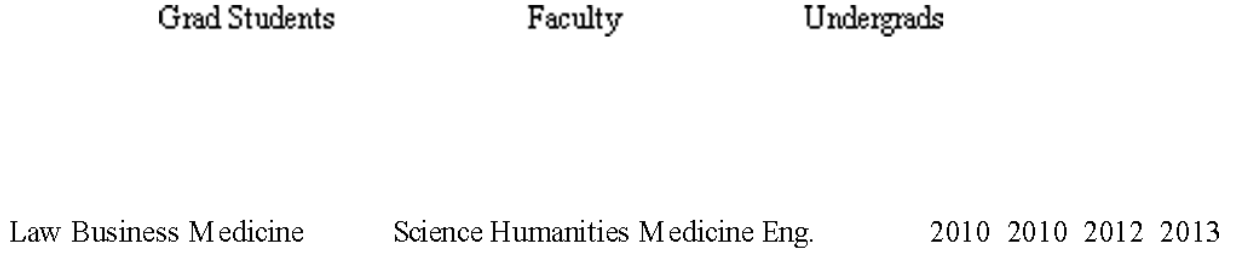


Figure 2.2: A possible multiscale decomposition of the Duke social structure

Since we would like to measure differences analytically, we associate with a graph G its markov transition matrix P . Our multiscale analysis procedure measures the changes in P as a proxy for the graph. The transition matrix is uniquely associated with a given graph up to permutations.

This paper is organized as follows: In Section 2.2, we introduce terminology and the multiscale analysis methodology. In Section 2.3, we analyze simulated graphs with multiscale structure. In Section 2.4, we analyze the evolution of the SP 500 over a 7 year period. We conclude with a discussion of future work in Section 2.5.

2.2 Multiscale Analysis Methodology

2.2.1 Notation

We define commonly used terms in the literature. A graph G is uniquely specified by its weight matrix W , where $W(i, j)$ denotes the weight between vertex i and j .

1. $D(i, i) = \sum_k W(i, k)$ and $D(i, j) = 0$ for $i \neq j$. Matrix of the row sums of the weights. This is used to normalize W .
2. $P = D^{-1}W$. Markov transition matrix for the graph. $P(i, j) = Pr(v_{k+1} = j | v_k = i)$ where v_k is a stochastic process with transition matrix P .
3. P^τ . Markov transition matrix to the τ power. $P^\tau(i, j) = Pr(v_{k+\tau} = j | v_k = i)$ where v_k is a stochastic process with transition matrix P .
4. $P_C^\tau = (D_C^{-1} \chi_C P^\tau \chi_C^T)$. This is a downsampling of the operator P^τ . χ is a projection onto the partition given by C . Multiplication by D_C^{-1} normalizes the row sums to be 1. This is the matrix of transition probabilities between the clusters after time τ . P_C has dimensions $|C| \times |C|$.
5. $P_C^\tau(C^i, C^j)$. Transition probabilities for the coarse-grained graph given by the partition C . This is the probability of moving from C^i to C^j after τ steps. This is given by the formula

$$P_C^\tau(C^i, C^j) = \sum_{y \in C^j} \frac{1}{|C^i|} \sum_{x \in C^i} P^\tau(x, y)$$

6. G_t is a time series of graphs indexed by t . P_t and other quantities indexed by t refer to the quantity for G_t e.g. P_t is the transition matrix for G_t .

7. $C_{j,t}$ is a multiscale family of partitions for the graph $G_t = (V_t, E_t)$. For all j ,

$$V_t = \sqcup_{C^i \in C_{j,t}} C_{j,t}^i$$

Furthermore, the partitions satisfy the property, $|C_{j,t}| > |C_{j-1,t}|$ for all j . We refer to $C_{j,t}$ as a partition of the graph. $C_{j,t}^i$ is a cluster within the partition $C_{j,t}$.

2.2.2 Random Walk on Graphs

Our main object of interest associated to a graph G_t is the transition matrix $P_t = D_t^{-1}W_t$. The matrix P_t has the interpretation of the transition probabilities of a random walk on G_t . We are concerned with studying the changes in P_t for a time series of graphs G_t . The dynamics of the graph time series are completely captured in the time series of transition matrices P_t . Thus we perform a multiscale analysis of the transition matrices P_t .

Numerous studies in the field of social networks, manifold learning and machine learning focus on studying the clusters within a network. A cluster is roughly defined as a set of nodes, V , that are highly connected within itself and sparsely connected to V^c . There are several ways to define the goodness of a clustering or partition, but one of the most pervasive ways to measure this is the Normalized Cut.

Definition 1. *The normalized cut objective of a partition $V = \sqcup_i C^i$ as*

$$Ncut(C^1, C^2, \dots, C^k) = \frac{1}{2} \sum_{i=1}^k \frac{W(C^i, (C^i)^c)}{vol(C^i)}$$

where $W(C^i, C^j) = \sum_{x \in C^i, y \in C^j} W(x, y)$ and $vol(C^i) = \sum_{x \in C^i} \sum_y W(x, y)$

The clustering problem is to find the partition of size k that minimizes the normalized cut objective. This problem is in general NP-hard and the best approximation ratio known to date is $O(\sqrt{\log n})$ due to Arora et al. [2] However, one relaxation of the k-clustering problem is to use spectral clustering on $P = D^{-1}W$.

We do not attempt to construct an optimal cut, instead we find clusters of vertices using suboptimal methods such as spectral clustering and online updating of the clusters. In fact, our algorithm does not require an optimal cluster; instead, nodes within the same clusters should stay clustered for some duration of the time series.

2.2.3 Downsampling the Graph

For each transition matrix P_t and partition at scale j , $C_{j,t}$, we construct downsampled operators $P_{t,C_{j,t}} = D_{C_{j,t}}^{-1} \chi_{C_{j,t}} P \chi_{C_{j,t}}^T$. Analogously, the downsampled operator of $P_t^{\tau_j}$ is $P_{t,C_{j,t}}^{\tau_j} = D_{C_{j,t}}^{-1} \chi_{C_{j,t}} P^{\tau_j} \chi_{C_{j,t}}^T$. The interpretation of the downsampled operator is the transition matrix for the coarsified graph. To coarsify a graph, each cluster $C_{j,t}^i$ is replaced with a node and connected to every other cluster with weight $W(C_{j,t}^i, C_{j,t}^l)$. The operator $P_{t,C_{j,t}}$ is the transition matrix of the coarsified graph. Thus for each graph G_t and multiscale partition, we construct the corresponding coarsifications of G_t . See Figure 2.3 for an example of a graph coarsification.

2.2.4 Diffusion Time

We introduce a new parameter that controls the diffusion time scale τ . $P^\tau(i, j)$ is the probability of transitioning from i to j in τ steps. Thus P^τ is an averaged version of P .

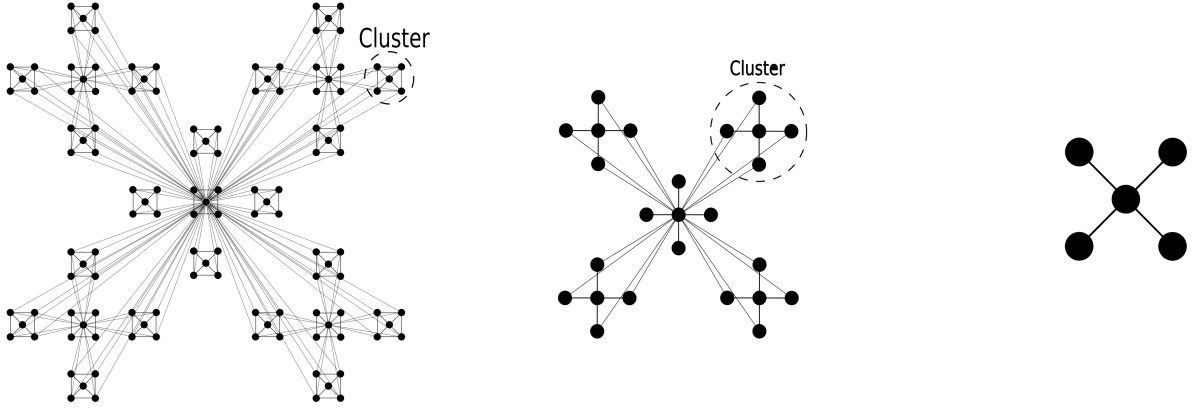


Figure 2.3: Graph Coarsification Procedure: from fine scale to coarse scale.

If there are many connections between i and j then $P^\tau(i, j)$ will be large for a range of τ . If i and j are in different clusters, but share an edge then $P(i, j)$ will be large, but $P^\tau(i, j)$ will be small. The τ parameter adds robustness to the method.

The τ parameter is also scale and graph dependent. For example when comparing two coarsified graphs, $P_{t, C_1}^{\tau_1} - P_{t-1, C_1}^{\tau_1}$ for the appropriate τ_1 is more meaningful than $P_{t, C_1} - P_{t-1, C_1}$. The latter quantity only reflects the change in the one step dynamic of the graph, so only the change in the immediate connectivity between the clusters affects this. However by exponentiating to the τ_1 , the change in the random walk after τ_1 steps is measured. This will negate the "boundary" effects that is measured and reflect the averaged change of the coarsified random walk. Similarly for the fine scale, $P_{t, C_2}^{\tau_2} - P_{t-1, C_2}^{\tau_2}$ is analyzed. In general, $\tau_2 < \tau_1$ because the random walk mixes faster on the finer scales. For $\tau \rightarrow \infty$, the difference measures the change in the stationary distribution of the coarsified graph which only depends on the degree of the vertices. Thus the goal is to choose a τ where the coarsified graph has sufficiently mixed to negate boundary effects, but has not reached the stationary distribution. For further discussion of the mixing time and coarsification of random walks see [6, 12].

2.2.5 Multiscale Measurements of the Dynamics

Since we have associated with each graph G_t , the transition matrix P_t we can measure changes as $P_t^\tau - P_{t-1}^\tau$ for some choice of τ . We note that the choice of τ is important because without the averaging effect of the diffusion time, noise may cause large changes. The entries with large changes represent clusters that exhibit interesting dynamics. We repeat the same process to compare the transition matrices at different scales which allows us to find the clusters at each scale that have interesting dynamics. The multiscale measurements of the error allow us to localize a change on the graph to one region/cluster of the original graph.

There are several different types of dynamics that we would like to study; these are illustrated in Figure 2.4. In the first row of Figure 2.4 from left to right is the original graph, a vertex joining, and a vertex leaving. In the second row of Figure 2.4 is an edge weight increasing, an entire new cluster joining the graph and two clusters merging together. Of course, these type of changes can happen at any given scale or even between scales. These are all example of dynamics that the multiscale analysis methodology will detect.

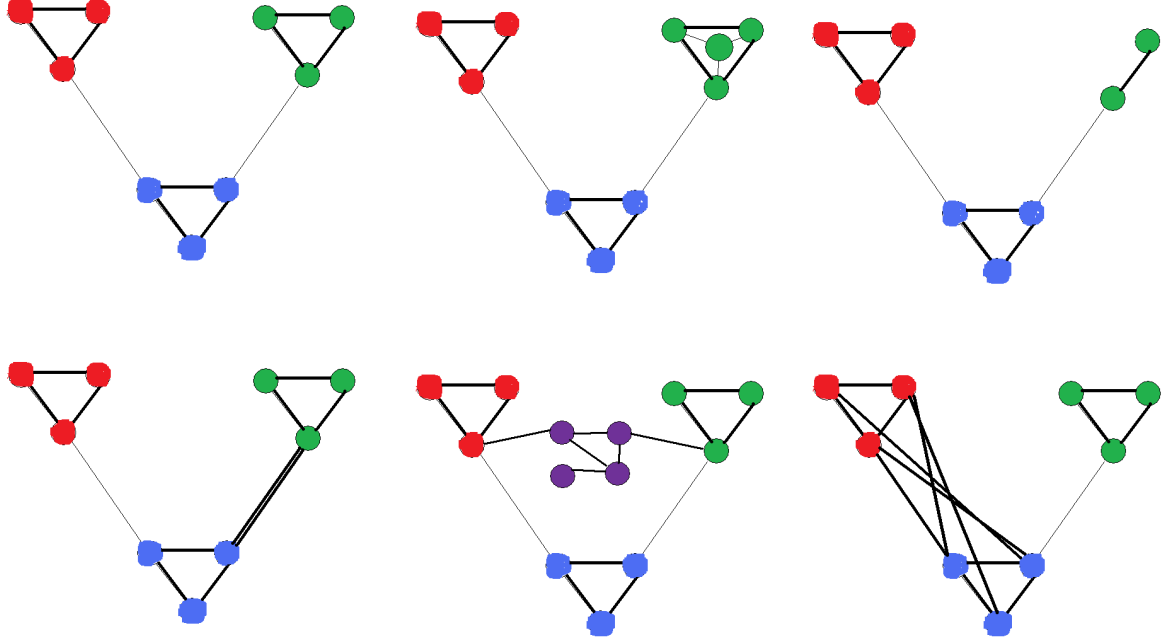


Figure 2.4: Possible different type of graph dynamics

2.2.6 Description of the Algorithm

Algorithm 2. For each time point t and scale j do the following:

1. Create a partition $C_{j,t}$ for G_t from the partition for G_{t-1} using the online update of the partition. If the partition does not perform well with respect to the Cut Objective, recompute the partitions using spectral clustering.
2. Compute coarse grained transition matrix: $P_{t,C_{j,t}}$
3. Measure the change in the graph time series: $\|P_{t,C_{j,t}}^{\tau_j} - P_{t-1,C_{j,t-1}}^{\tau_j}\|$

2.3 Simulation Results

We test the multiscale analysis algorithm on a simulated graph time series. The purpose of the simulation is to illustrate that our methodology can adequately detect several types of dynamics. The first type of dynamics is vertices leaving and entering. Next, the edge weights on the graph will change between each graph. The dynamics translate to changes on different scales depending on the type of dynamics. For example, two fine scale clusters merging will cause a large change in the fine scale. We show that the algorithm described in Section 2.2 can effectively identify these dynamics.

2.3.1 Construction of Graphs

The simulated graphs have clear multiscale structure. The edges on the finest scale are the largest and decrease by a factor of 8 each scale as we move to coarser scales. Thus

the graph has a clear clustering into 3 clusters on the coarsest scale, 15 clusters on the next scale and finally 60 clusters on the finest scale. See Figure 2.5a for the simulated multiscale graph.

2.3.2 Adding Vertices and Edges

We introduce new vertices in a random fashion. The first neighbor of the new vertex is chosen at random over the entire graph. The vertex then chooses his edge weights according to a fixed distribution with larger weights assigned a smaller probability. The remaining neighbors are chosen with the distribution $F(v) = \alpha F_{diff}(v) + (1 - \alpha) + Un(v)$ where F_{diff} is the probability of the introduced vertex of diffusing to another vertex after time τ and $Un(v)$ is the uniform distribution over the vertices.

We also add edge noise to all the edge weights. The edge noise has distribution $N(0, \sigma_{ij})$ where $\sigma_{ij} = \frac{W(i,j)}{10}$.

2.3.3 Updating of the Clusters

We use an online method to determine the cluster assignments of the vertex z that is joining the graph. We find the cluster assignment that minimizes 1 without changing the assignments of any other vertices. Let C represent the partition prior to z joining and $C_{z,l}$ represent the partition after vertex z joins C^l . Let $J(C, \tau) = \sum_{i=1}^k P^\tau(C^i, C^i)$. This is not equivalent to minimizing the normalized cut objective, but we find this objective is more well-suited to identifying dynamics. We try to maximize $J(C_{z,l}, \tau)$ for some choice of l .

$$J(C_{z,l}, \tau) = \sum_{i \neq l} P^\tau(C^i, C^i) + \frac{1}{|C^l| + 1} \sum_{x \in C^l} [P^\tau(x, z) + P^\tau(z, x) + P^\tau(z, z)] \quad (2.1)$$

$$+ \frac{1}{|C^l| + 1} \sum_{x \in C^l, y \in C^l} P^\tau(x, y) \quad (2.2)$$

$$= J(C, \tau) - \frac{1}{|C^l|} \sum_{x \in C^l, y \in C^l} P^\tau(x, y) \quad (2.3)$$

$$+ \frac{1}{|C^l| + 1} \sum_{x \in C^l, y \in C_l} P^\tau(x, y) + \frac{1}{|C^l| + 1} \sum_{x \in C^l} [P^\tau(x, z) + P^\tau(z, x) + P^\tau(z, z)]. \quad (2.4)$$

Since we are maximizing over choice of C^l , we compute the terms after $J(C)$ for each l and find the largest. We assign z to the C^l that maximizes $J(C_{z,l}, \tau)$.

2.3.4 Description of the Experiment

A time series of 51 graphs was constructed with a single vertex and edge noise added after each graph. The initial graph has 3 layers, and 60 nodes. The edge weights on the scales from finest to coarsest are 64, 8, and 1. The initial partitions are fed to the algorithm and we update the partitions using the aforementioned online algorithm. At each step, a new vertex with 3 edges is introduced.

The change in the graph time series is computed as the frobenius norm between $P_{C_{j,t},t}^{\tau_j} - P_{C_{j,t-1},t-1}^{\tau_j}$ for each scale j and time t . The partition $C_{j,t}$ is updated using the online algorithm since the number of nodes in each G_t changes. The partition is recomputed

when there is a large change on the coarse scale, since this indicates these partitions no longer resemble good clusterings. We recompute the clusterings using spectral clustering. See [17] for details.

2.3.5 Analysis of the Dynamics

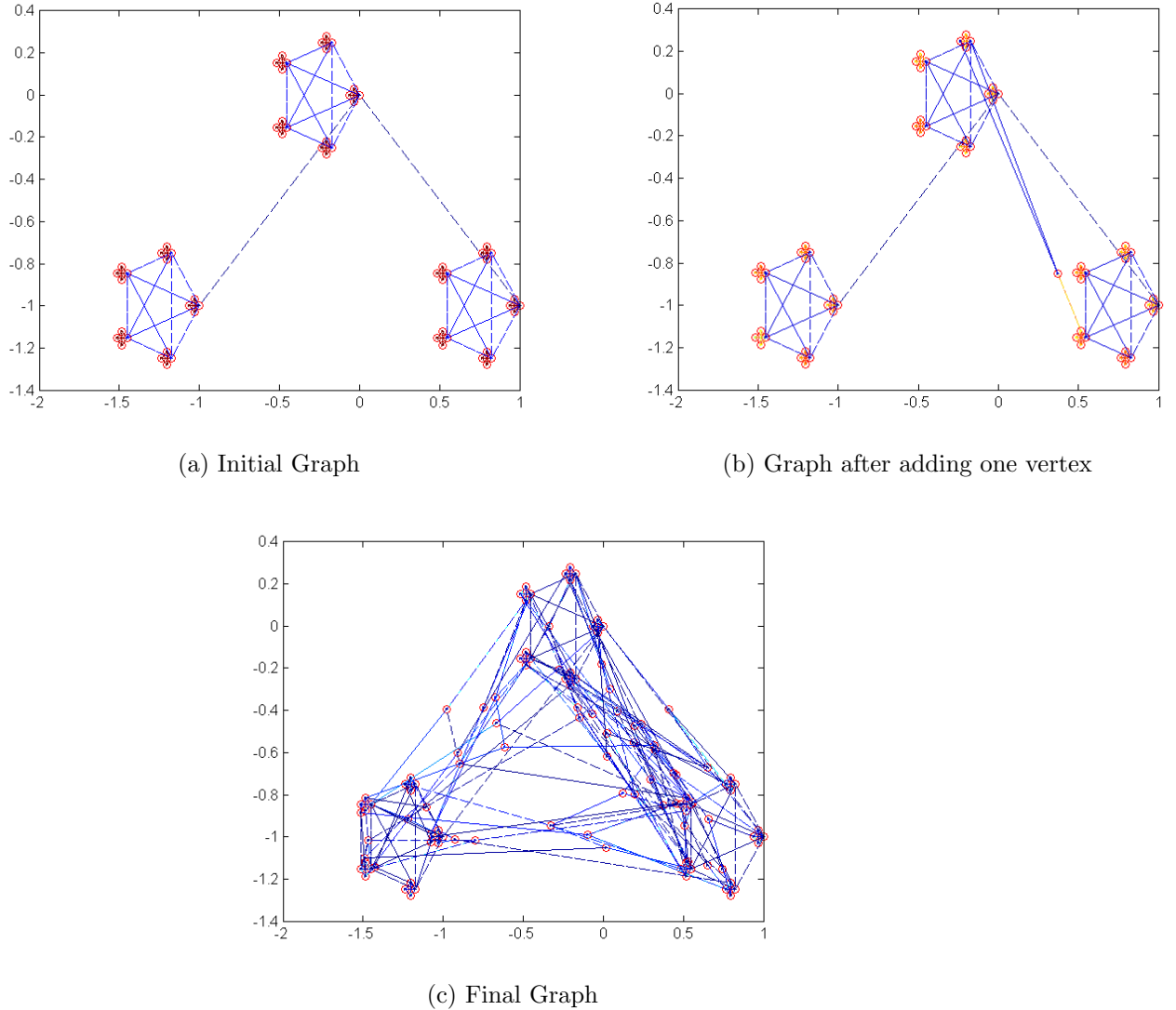


Figure 2.5: Multiscale Graphs at different times.

In this example, the graph has two different scales. The coarse scale consists of the 3 large clusters. The finer scale consists of 15 clusters. For this simulation, we fix the number of clusters at 2 and fix 3 clusters at the coarse scale and 15 clusters at the fine scale. After several perturbations, the resulting graph may no longer have 3 coarse scale clusters or 15 fine scale clusters, but the dynamics can still be effectively measured by fixing the number of scales and clusters.

As stated before, the change in the graph is measured as $\|P_{C_{1,t},t}^{\tau_1} - P_{C_{1,t-1},t-1}^{\tau_1}\|_F$ and $\|P_{C_{2,t},t}^{\tau_2} - P_{C_{2,t-1},t-1}^{\tau_2}\|_F$. This is plotted in Figure 2.8. We update $C_{j,t}$ using the online updates and recompute $C_{j,t}$ when there is a large change on the coarse scale. The plot corresponding to the change in the fine scale is much more jittery. The minimum value

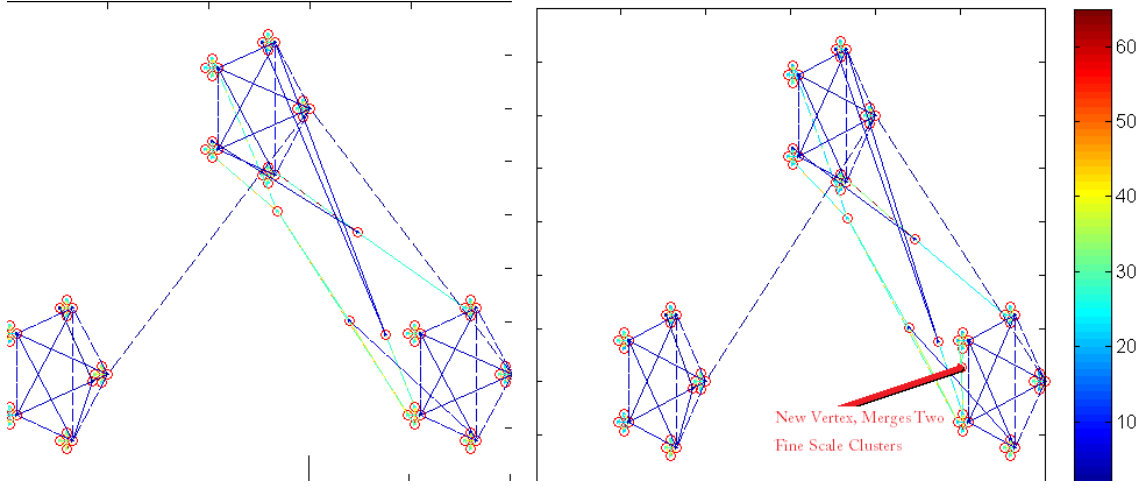


Figure 2.6: A vertex joining causes two fine scale clusters to merge. This is only seen in the fine scale measurements.

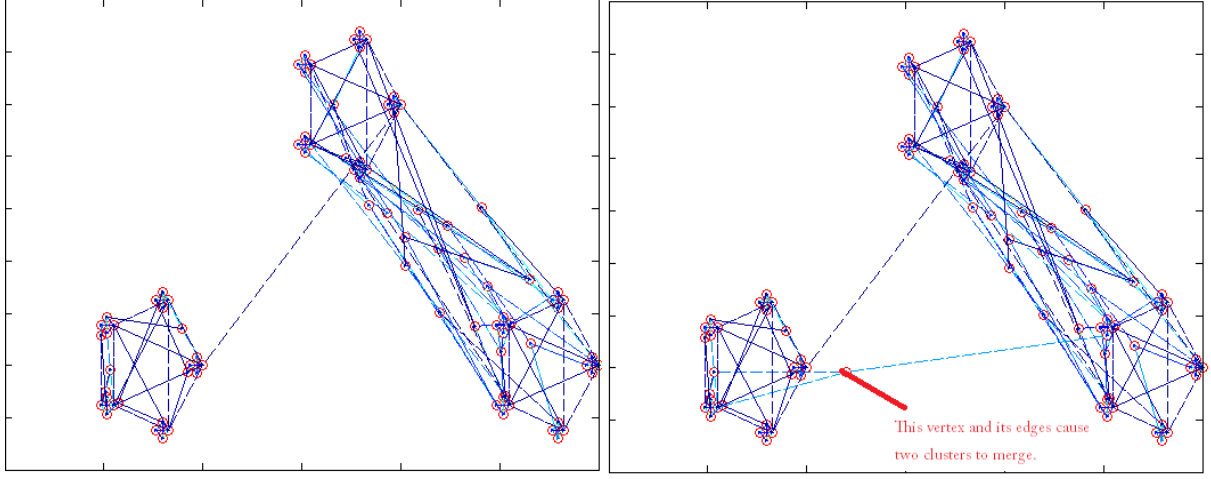


Figure 2.7: A vertex merges two coarse scale clusters. Notice the vertex in the plot on the right, but not in the left side plot. This vertex causes a large change between these two graphs since it joins two coarse scale clusters.

it attains is near .1. This is because even small changes in the graph are altering the fine scales. However, the plot of change in the coarse scale takes on much lower values. The large peaks in the coarse scale plot are also large peaks in the fine scale plot. On the other hand, many of the peaks in the fine scale plot are either much smaller or disappear in the coarse scale plot. For example, the 7th time point causes a large change in the fine scale, but almost no change on the coarse scale (Figure 2.8). In fact, we see that the difference between G_7 and G_6 is a vertex with edges to two fine scale clusters joined and thus merged two fine scale clusters. This does not affect the coarse scale since the newly introduced edges are all within the same coarse scale cluster. See Figure 2.6 for details.

The peak at time point 2 in Figure 2.8 is due to a vertex joining with edges between two coarse scale clusters. This can be seen by comparing Figures 2.5a and 2.5b. This change is evident on both scales since it causes two coarse scale clusters to merge, and also causes two fine scale clusters to join.

The largest change is at time point 27. Before time 27, two of the coarse clusters are tightly interconnected, but one cluster is not connected to the other two. The vertex entering at time 27 causes the lone cluster to be connected to the other two. This causes

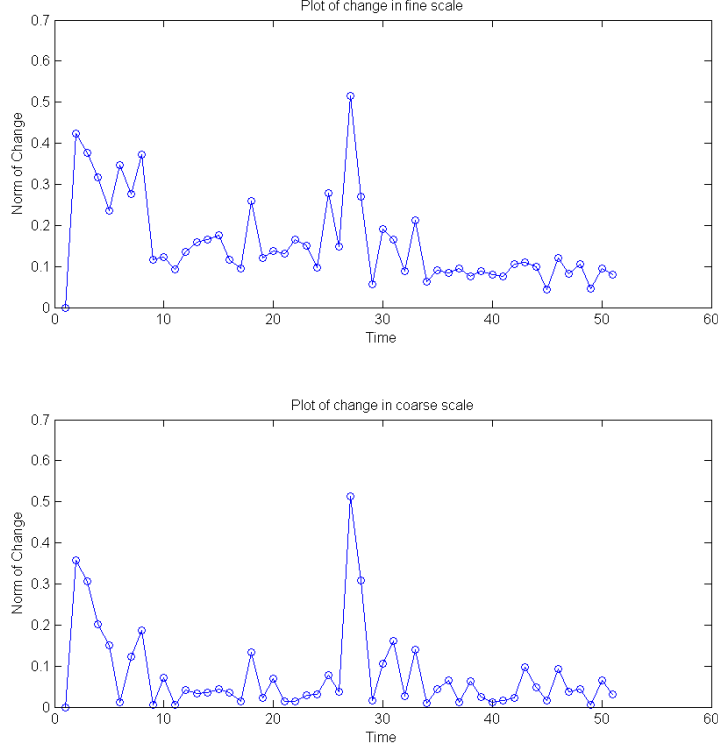


Figure 2.8: Plot of the change between G_t and G_{t-1} . Y-axis is $\|P_{C_{j,t},t}^\tau - P_{C_{j,t-1},t-1}^\tau\|_F$.

the large change at time 27 that is seen in Figure 2.8. See Figure 2.7 for the change between G_{26} and G_{27} .

2.3.6 Interplay between Diffusion Time and Dynamics

In Figures 2.9a, 2.9b and 2.9c, we plot the magnitude of $\|P_{t,C_{1,t}}^\tau - P_{t-1,C_{1,t-1}}^\tau\|$ and $\|P_{t,C_{2,t}}^\tau - P_{t-1,C_{2,t-1}}^\tau\|$ for a large range of τ . In the first row τ varies from 1 to 200 in increments of 1, in the second row τ varies from 1 to 2000 in increments of 10 and in the third row τ varies from 1 to 150000 in increments of 3000.

The most obvious observation is that the fine scale plots take on larger values than the coarse scale plots. As expected, the fine scale is more sensitive to perturbations and noise added to the graph time series. In fact, many of the entries of the coarse scale plot are near 0, yet they affect the fine scale. From these plots, it is also evident that for smaller τ the coarse scale plots are not very informative. In the first row on the right for $\tau < 25$, the plot indicates almost no change between the graph time series. On the other hand, the corresponding fine scale plot (first row on the left) reveals dynamics of the time series.

In Figure 2.9c, we notice that the measured change is the change in the stationary distribution of the coarsified graphs. The stationary distribution is only affected by the degrees of the vertices. The stationary distribution change of the fine scale is larger because the change is relative to the weight of the cluster and the weight of the edges being introduced. Since the size/weight of the fine scale clusters are smaller, they are more influenced by vertices with heavy edges joining. At time point 27, a vertex joined that merged two coarse clusters. This caused the dark red band seen in Figure 2.9b, but the magnitude of the change decays as τ increases because the stationary distribution was

not altered. At time point 6, a vertex joined with large edge weights but did not merge any clusters. This did not register as a change for the small and mid-range values of τ on the coarse scale plot; in Figure 2.9c, we notice that at time point 6 the perturbation does alter the stationary distribution because of the large edge weights.

2.4 Experimental Results

In this section, we analyze the SP 500 from January 2002 until February 2009. During this period, there are 478 stocks that are within the SP 500 for the entire period.

2.4.1 Building the Graph

The graph time series is built on top of the percentage returns of the stocks of the SP 500. Each node in the graph is associated with a particular stock and the edges represent stocks that performed similarly during that period. Stocks that perform similarly will be connected high edge weights; the specifics are discussed below.

From the percentage returns of each stock, two graph time series are built G_t and $\tilde{G}_t$. Each Graph G_t is built from a feature matrix X_t with dimensions 478×6 where $X_{t,ij} = \%$ return of stock i over month $t + j - 1$. The rows of X_t give the returns of stock i over a 6 month period. For the purpose of this experiment, a month is 20 trading days. A graph is then constructed using X_t through the procedure in [24]. Each vertex is connected to its 20 nearest neighbors with weight $\exp - \frac{\|x_i - x_j\|^2}{2\sigma_i^2}$. σ_i is chosen to be $\|x_i - x_i^{10}\|^2$ where x_i^{10} is the 10th nearest neighbor of x_i . The parameters of 10 and 20 are arbitrarily chosen, but were found to work best. The weights are then symmetrized

by defining $W_{t,ij} = \frac{\exp - \frac{\|x_i - x_j\|^2}{2\sigma_i^2} + \exp - \frac{\|x_j - x_i\|^2}{2\sigma_j^2}}{2}$, where x_i and x_j are the i^{th} and j^{th} rows of X_t . The weight matrix W_t specifies an undirected graph where weights are given by a similarity measure of stock performance.

The graph time series $\tilde{G}_t$ is built in a similar fashion except $\tilde{X}_t$ is a matrix with dimensions 478×5 and $\tilde{X}_{t,ij} = \%$ return of stock i over month $t + j$. Thus $\tilde{G}_t$ is a graph representing the stock market from months $t + 1$ to $t + 5$ and G_t represents the stock market from months t to $t + 5$.

The algorithm is summarized below:

Algorithm 3 (Graph Construction Algorithm). *For each t do the following,*

1. *Construct X_t the matrix of percentage returns over a six month period. $X_{t,ij} = \%$ return of stock i over month $t + j - 1$.*
2. *Build graph using X_t as the feature matrix. Let $W_{t,ij} = \frac{1}{2}(\exp - \frac{\|x_i - x_j\|^2}{2\sigma_i^2} + \exp - \frac{\|x_j - x_i\|^2}{2\sigma_j^2})$ with $\sigma_i = \|x_i - x_i^{10}\|$ where x_i^{10} is the 10th nearest neighbor of x_i .*

2.4.2 Multiscale Analysis Algorithm

We adapt the algorithm provided in Section 2.2.6 to this graph time series. The primary difference is that the multiscale partitions we use will be hierarchical and dyadic. This means that at scale j , the partition contains 2^j clusters and the clusters are subsets of a cluster in scale $j - 1$. This can be done efficiently by bisecting at the median of the 2nd eigenvector.

Algorithm 4 (Multiscale Analysis of Stocks). *For each time point t and scale j do the following:*

1. *Create a partition $C_{j,t}$ for $\tilde{G}_t$ using hierarchical spectral clustering.*
2. *Compute coarse grained transition matrices for $\tilde{G}_t$: $\tilde{P}_{t,C_{j,t}}$.*
3. *Compute coarse grained transition matrices for G_{t+1} : $P_{t+1,C_{j,t}}$*
4. *Measure the change in the graph time series: $\|\tilde{P}_{t,C_{j,t}}^{\tau_j} - P_{t+1,C_{j,t}}^{\tau_j}\|$*

The expression $\|\tilde{P}_{t,C_{j,t}}^{\tau_j} - P_{t+1,C_{j,t}}^{\tau_j}\|$ measures how different month $t+6$ is from the 5 months preceding it.

2.4.3 Analysis

For the stock market, we study the same plots used in Section 2.3.5. In Figure 2.11a, the change in the graph is plotted at scales 1 and 3. The most striking feature of these plots is that the scale 1 and 3 plots are very similar. Unlike in the case of the simulations, the peaks and valleys in both plots occur in the same places. If there were changes in the market that only affect one sector, we would expect that the fine scale plot exhibit many changes where the coarse scale plot is unaffected. It is difficult to determine why the two plots are so similar, but a likely explanation is the chosen clusters are not meaningful. If stocks in the same clusters do not behave similarly over time, then the plots at finer scales will not detect changes that are affecting a specific sector.

In Figure 2.11b, the monthly percentage change in the SP500 over that time is plotted. This plot is very different from the plot in Figure 2.11a. This shows that our algorithm is detecting changes that cannot be seen from just following the SP500 index. The large change at time 79 on both plots corresponds to the market correction in September 2008. This is also seen in Figure 2.10. The large change at time 64 corresponds to around July 2007 when the market reached a record high.

Figure 2.12 shows the amount of change for a given τ and time on scales 1 and 3. From these plots, it is easy to identify the value of τ for a given scale. The proper value of τ is where the change reaches a local maxima. In scale 1, this value is around 6; in scale 3, this value is around 2. The observations made for the simulation graphs also hold for these plots. For values of τ that are too small, the changes are too small and too noisy. For values of τ too large, the changes only reflect the stationary distribution. The appropriate choice of τ can be found from these plots by identifying a range where the changes exhibit a local maximum.

We are unable to explain many of the peaks and troughs in Figure 2.11a. The plot is very noisy, but this might be due to the volatile nature of the stock market. In the future, the same algorithm will be tested on mutual funds which have been shown to cluster well and thus a more amenable dataset for our methodology. Nevertheless, this experiment demonstrates the versatility of multiscale analysis of dynamic graphs on data that is not traditionally thought of as a graph.

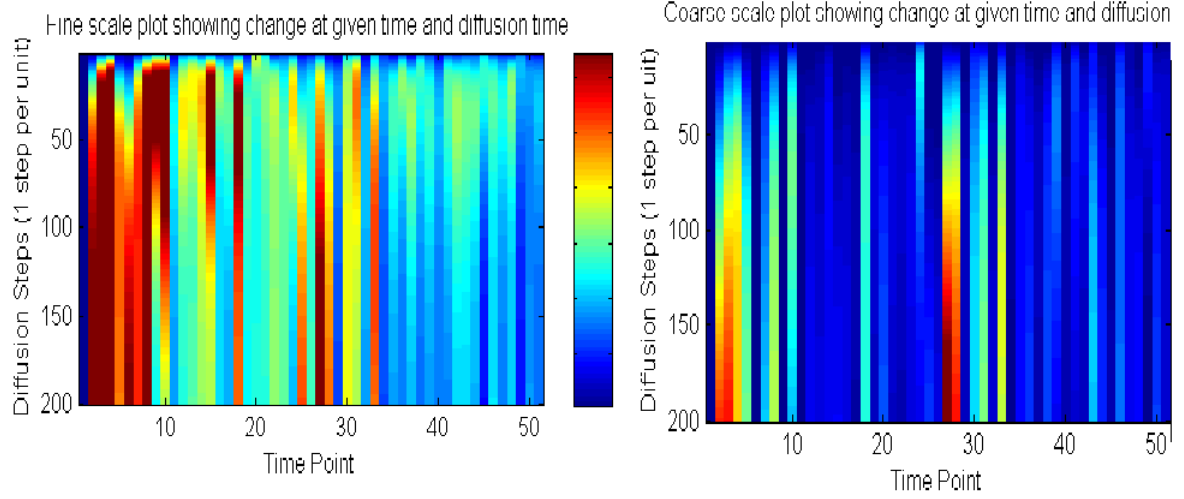
2.5 Future Work

The proposed algorithm is still in the preliminary stages. There are still several portions that need to be refined. The first problem is finding the partitions/clusters efficiently

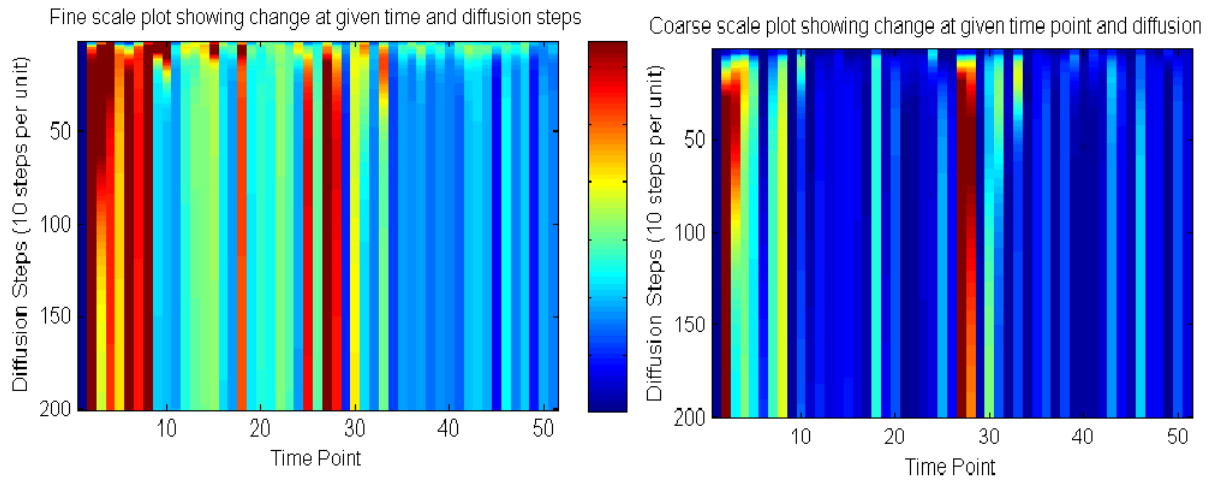
and checking that they are meaningful. For some social networks such as Facebook, the users are already classified into clusters by their networks and groups. However for most networks, the clusters have to be identified using a clustering algorithm. We chose to use hierarchical spectral clustering because of its simplicity; in the future, we may adapt METIS [11] which provides dyadic decompositions.

Identifying clusters is known to be a very difficult problem and computationally expensive. This leads to the second issue of finding clusters for the entire graph time series without recomputing partitions each time. Instead, the partitions should be updated for each member of the graph time series. This problem is non-trivial because the edges and nodes are different between two instances of the same graph. For the simulations, we choose a greedy online update that does not modify the cluster assignment of any existing nodes. A more intelligent algorithm can be made by choosing an initial guess for a partition and then using a local search algorithm such as the classical Kernighan-Lin to refine the partition. More recent work on cut refinement includes [18] and references therein.

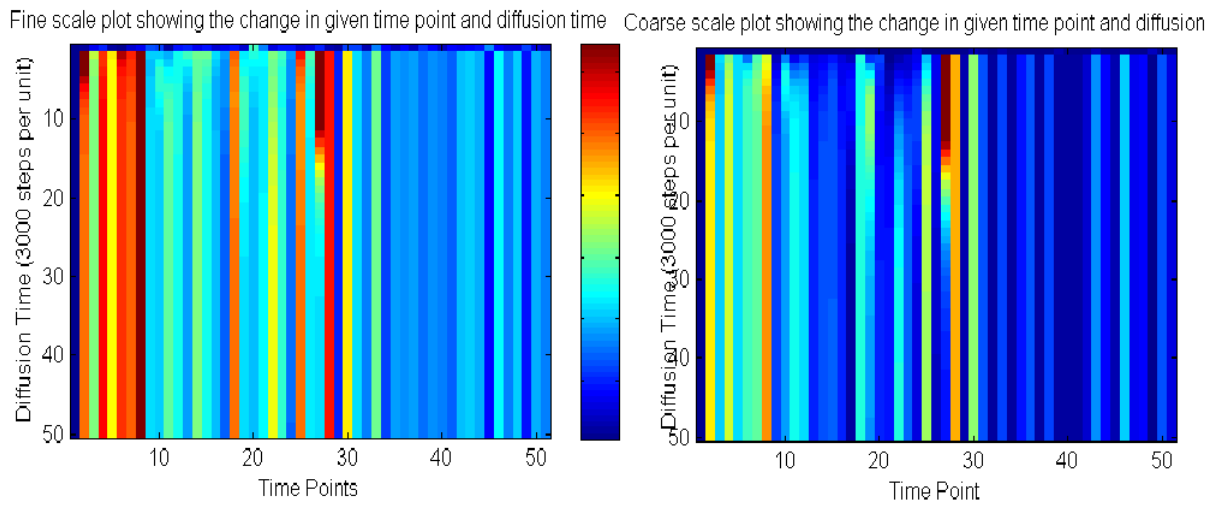
In addition to identifying the time which changes occur, we wish to identify the nodes or clusters causing these changes. Many changes are expected to be localized or caused due to a cluster or a small number of nodes. Using a multiscale decomposition of $P_t - P_{t+1}$, we are designing a procedure to isolate the dynamics and identify whether dynamics are due to particular clusters or distributed throughout the network. This procedure resembles wavelet methods for time series analysis.



(a) Magnitude of Change for Small τ



(b) Magnitude of Change for Mid τ

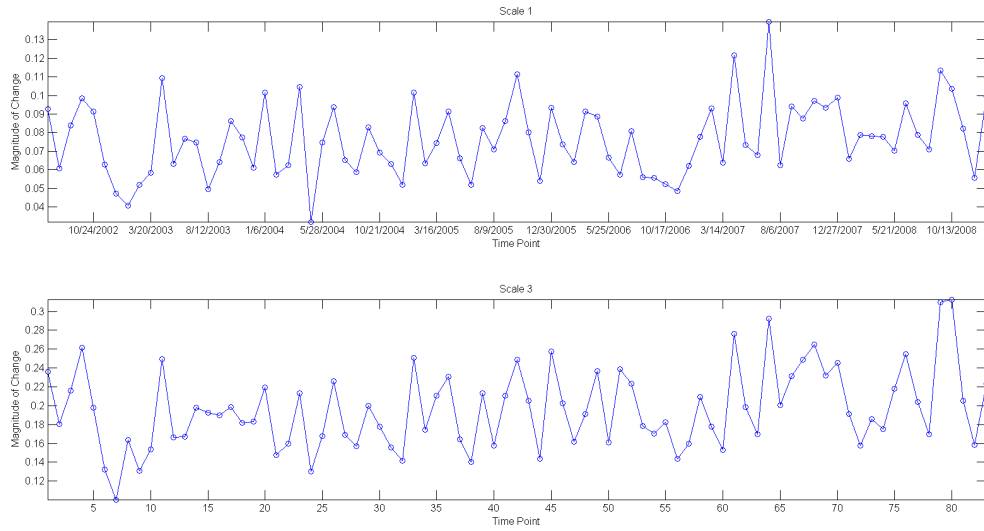


(c) Magnitude of Change for large τ

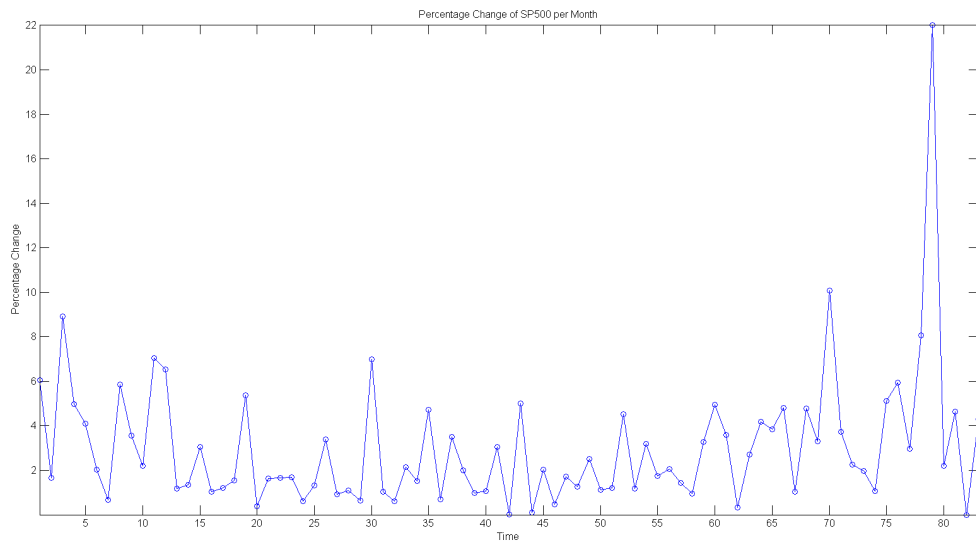
Figure 2.9: Plot of amount of change at given value of τ and t .



Figure 2.10: Plot of SP500 index from January 2002 until February 2009.



(a) Plot of dynamics for scale 1 and scale 3



(b) Percentage Change of SP500 per Month

Figure 2.11: Top plot shows the dynamics on scale 1 and 3. Bottom plot shows the percentage change of the SP500 over the same time frame.

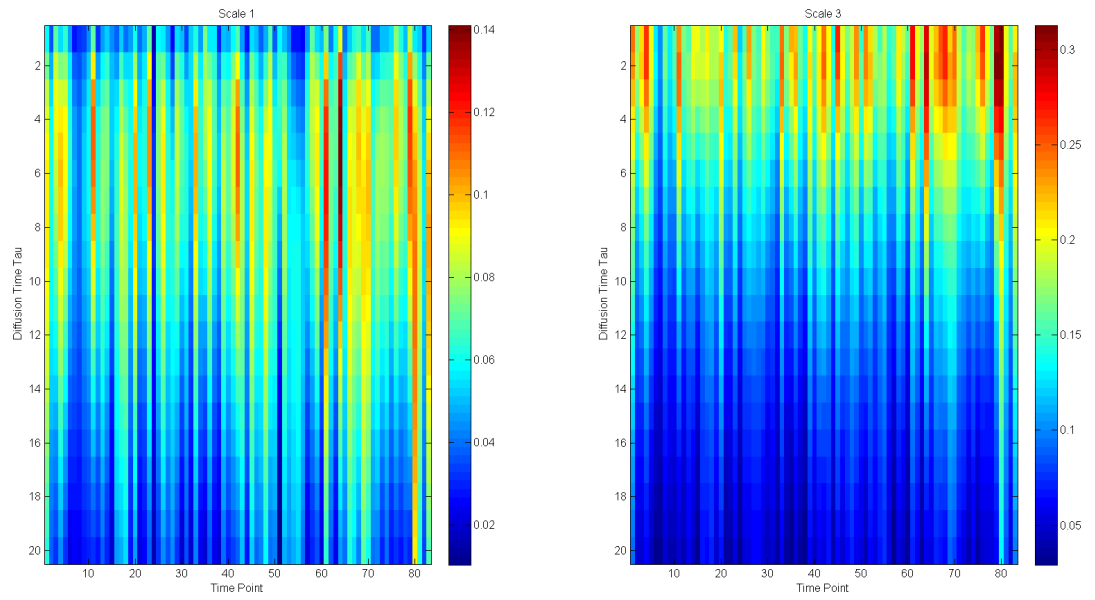


Figure 2.12: Plot of amount of change at given τ and time for scales 1 and 3.

Chapter 3

Acknowledgements

I would like to thank my advisor, Mauro Maggioni, for his guidance and time on both of these projects. Mauro is full of interesting ideas, insights and enthusiasm. He was always available to help on all aspects of research and available to answer questions. It would be difficult to overstate how much I have learned from him.

I would like to thank Bill Allard for taking the time to read this thesis. Without the efforts of David Kraines to run the PRUV fellow program, none of this would have been possible. I also benefited from many lunches and dinners at his house with the other PRUV fellows! Much of the work on dimension estimation in Chapter 1 is due to work with Anna Little and Mauro Maggioni. I also discussed several parts of this thesis with Guangliang Chen. Guangliang was always available to talk about anything research-related and greatly contributed to my understanding.

Bibliography

- [1] Ery Arias-Castro, Guangliang Chen, and Gilad Lerman, *Spectral clustering based on local linear approximations*, (2010).
- [2] Sanjeev Arora, Satish Rao, and Umesh Vazirani, *Expander flows, geometric embeddings and graph partitioning*, Journal of the ACM **56** (2009), no. 2, 1–37.
- [3] A.L. Barabási and R. Albert, *Emergence of scaling in random networks*, Science **286** (1999), no. 5439, 509.
- [4] K. Carter, A. O. Hero, and R. Raich, *De-biasing for intrinsic dimension estimation*, Statistical Signal Processing, 2007. SSP '07. IEEE/SP 14th Workshop on (2007), 601–605.
- [5] K.M. Carter and A.O. Hero, *Variance reduction with neighborhood smoothing for local intrinsic dimension estimation*, Acoustics, Speech and Signal Processing, 2008. ICASSP 2008. IEEE International Conference on (2008), 3917–3920.
- [6] RR Coifman and S Lafon, *Diffusion maps*, Applied and Computational Harmonic Analysis (2006).
- [7] J.A. Costa and A.O. Hero, *Geodesic entropic graphs for dimension and entropy estimation in manifold learning*, Signal Processing, IEEE Transactions on **52** (2004), no. 8, 2210–2221.
- [8] K. Fukunaga and D.R. Olsen, *An algorithm for finding intrinsic dimensionality of data*, I.E.E.E. Trans. on Computer **C-20** (1971), no. 2.
- [9] G. Haro, G. Randall, and G. Sapiro, *Translated poisson mixture model for stratification learning*, Int. J. Comput. Vision **80** (2008), no. 3, 358–374.
- [10] Iain M. Johnstone, *On the distribution of the largest eigenvalue in principal components analysis*, Ann. Stat. (2001).
- [11] G. Karypis and V. Kumar, *METIS: Unstructured graph partitioning and sparse matrix ordering system*, The University of Minnesota **2**.
- [12] Stéphane Lafon and Ann B Lee, *Diffusion maps and coarse-graining: A unified framework for dimensionality reduction, graph partitioning, and data set parameterization.*, IEEE Transactions on Pattern Analysis and Machine Intelligence **28** (2006), no. 9, 1393–1403.
- [13] J. Leskovec, D. Chakrabarti, J. Kleinberg, C. Faloutsos, and Z. Ghahramani, *Kronecker graphs: An approach to modeling networks*, stat **1050** (2009), 21.

- [14] E. Levina and P. Bickel, *Maximum likelihood estimation of intrinsic dimension*, In Advances in NIPS 17, Vancouver, Canada (2005).
- [15] A.V. Little, J. Lee, Y.-M. Jung, and M. Maggioni, *Estimation of intrinsic dimensionality of samples from noisy low-dimensional manifolds in high dimensions with multiscale svd*, to appear in Proc. S.S.P. (2009).
- [16] A.V. Little, M. Maggioni, and L. Rosasco, *Multiscale geometric analysis: Estimation of intrinsic dimension*, in preparation (2010).
- [17] Ulrike Luxburg, *A tutorial on spectral clustering*, Statistics and Computing **17** (2007), no. 4, 395–416.
- [18] Michael W. Mahoney, Lorenzo Orecchia, and Nisheeth K. Vishnoi, *A Spectral Algorithm for Improving Graph Partitions*, (2009), 14.
- [19] M.E.J. Newman, A.L. Barabasi, and D.J. Watts, *The structure and dynamics of networks*, Princeton Univ Pr, 2006.
- [20] Debashis Paul, *Asymptotics of sample eigenstructure for a large dimensional spiked covariance model*, Statistica Sinica **17** (2007), 1617–1642.
- [21] P.W. Jones, *Rectifiable sets and the traveling salesman problem*, Inventiones Mathematicae **102** (1990), 1–15.
- [22] M. Raginsky and S. Lazebnik, *Estimation of intrinsic dimensionality using high-rate vector quantization*, Proc. NIPS (2005), 1105–1112.
- [23] Jack Silverstein, *On the empirical distribution of eigenvalues of large dimensional information-plus-noise type matrices*, Journal of Multivariate Analysis **98** (2007), 678–694.
- [24] Lihi Zelnik-manor, *Self-tuning spectral clustering*, Advances in neural information processing systems **17** (2004), no. 1601-1608, 16.